



Instituto Tecnológico de Mazatlán

Control de acceso con RFID por SPI

Carrera:

Ingeniería en Sistemas Computacionales

Materia:

Sistemas Programables

Profesor:

Miguel Hernández Barrón

Alumnos:

Rosa Guadalupe Romero Navarro

Javier Ramón Quintero Valdez

Gabriel Alejandro Barrón Sarabia

Reporte de práctica: Control de acceso con RFID por SPI

1. Descripción del proyecto

Esta práctica consiste en implementar un sistema de control de acceso utilizando un módulo lector RFID RC522, comunicado con el Arduino UNO R4 WiFi mediante el bus SPI (Serial Peripheral Interface).

Al acercar una tarjeta o llavero RFID al lector, el Arduino obtiene su UID (identificador único) y lo compara contra un UID autorizado previamente almacenado en el código. Si coincide, se enciende un LED verde indicando acceso permitido; si no coincide, se enciende un LED rojo indicando acceso denegado, sin bloquear en ningún momento la lectura continua del lector.

2. Objetivos

- Comprender el funcionamiento del protocolo de comunicación SPI.
- Configurar y utilizar un módulo lector RFID RC522 mediante la librería MFRC522.
- Leer e identificar el UID de una tarjeta o llavero RFID.
- Implementar una lógica de comparación para otorgar o denegar acceso según el UID leído.
- Controlar la temporización de los LEDs indicadores sin utilizar delay(), empleando millis().
- Diagnosticar fallas de comunicación SPI mediante la lectura del registro de versión del chip.

3. Herramientas y material utilizado

Hardware:

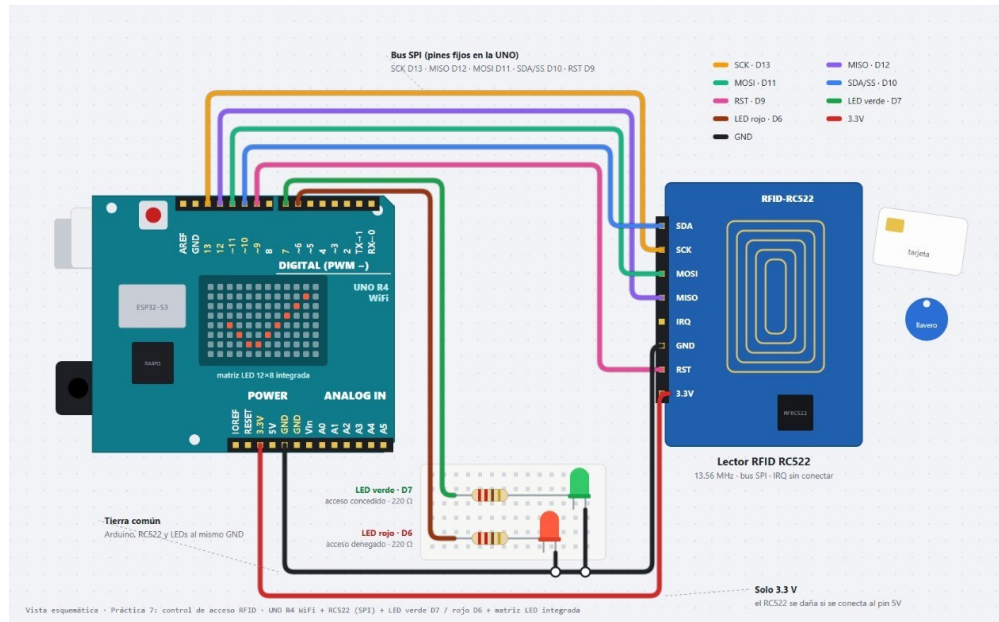
- Arduino UNO R4 WiFi
- Módulo lector RFID RC522 (SPI)
- Tarjeta y llavero RFID de prueba
- 2 LEDs (verde y rojo) con resistencia de 220Ω cada uno
- Protoboard
- Cables de conexión

Software:

- Arduino IDE
- Librerías SPI y MFRC522
- Monitor serie

4. Diagrama

El diagrama muestra la conexión entre el Arduino UNO R4 WiFi, el módulo lector RC522 y los dos LEDs indicadores. El RC522 se alimenta con 3.3V (nunca con 5V, ya que el chip se dañaría) y comparte con el Arduino el bus SPI: SCK en el pin 13, MOSI en el pin 11, MISO en el pin 12, SDA/SS en el pin 10 y RST en el pin 9. El LED verde se conecta al pin 7 y el LED rojo al pin 6, cada uno con su resistencia de 220Ω hacia GND.



5. Código

El siguiente código inicializa el módulo RC522, verifica que la comunicación SPI esté funcionando correctamente, y en cada ciclo revisa si hay una tarjeta presente para leer y validar su UID:

$$/^{*}$$

Control de acceso con RFID RC522 (SPI) - Arduino UNO/Nano

CONEXIONES (RC522 -> Arduino) ¡ALIMENTAR CON 3.3 V, NO 5 V!

```
SDA (SS)  -> D10
SCK        -> D13
MOSI       -> D11
MISO       -> D12
RST        -> D9
3.3V       -> 3.3V
GND        -> GND
IRQ        -> sin conectar
```

LEDs (cada uno con resistencia de 220 ohm a GND):

LED verde -> D7
LED rojo -> D6

* /

```
#include <SPI.h>
#include <MFRC522.h>
```

```
// ----- Pines -----  
const byte SS_PIN = 10;  
const byte RST_PIN = 9;  
const byte LED_VERDE = 7;  
const byte LED_ROJO = 6;
```

```
// ----- Configuración -----  
const unsigned long TIEMPO_LED = 2000; // ms que permanece encendido el LED
```

```
// UID autorizado: CAMBIA estos bytes por el UID de tu tarjeta
// (léelo primero con el Monitor Serie)
const byte UID_AUTORIZADO[] = {0x01, 0x35, 0xD5, 0x1C};
const byte TAM_UID AUTORIZADO = sizeof(UID AUTORIZADO);
```

```

MFRC522 rfid(SS_PIN, RST_PIN);

// ----- Control de tiempo con millis() -----
bool verdeEncendido = false;
bool rojoEncendido = false;
unsigned long inicioVerde = 0;
unsigned long inicioRojo = 0;

// Verifica comunicación leyendo el registro de versión del chip
bool lectorResponde() {
    byte version = rfid.PCD_ReadRegister(MFRC522::VersionReg);
    Serial.print(F("VersionReg = 0x"));
    Serial.println(version, HEX);
    // 0x00 o 0xFF significa que no hay comunicación SPI (ej. MISO desconectado)
    return (version != 0x00 && version != 0xFF);
}

void imprimirUID(const MFRC522::Uid &uid) {
    Serial.print(F("UID:"));
    for (byte i = 0; i < uid.size; i++) {
        Serial.print(' ');
        if (uid.uidByte[i] < 0x10) Serial.print('0');
        Serial.print(uid.uidByte[i], HEX);
    }
    Serial.println();
}

bool uidAutorizado(const MFRC522::Uid &uid) {
    if (uid.size != TAM_UID_AUTORIZADO) return false;
    return memcmp(uid.uidByte, UID_AUTORIZADO, TAM_UID_AUTORIZADO) == 0;
}

void setup() {
    Serial.begin(9600);
    while (!Serial) { ; }

    pinMode(LED_VERDE, OUTPUT);
    pinMode(LED_ROJO, OUTPUT);
    digitalWrite(LED_VERDE, LOW);
    digitalWrite(LED_ROJO, LOW);

    SPI.begin();
    rfid.PCD_Init();
    delay(50);

    if (lectorResponde()) {
        Serial.println(F("Lector RC522 detectado: comunicacion SPI OK"));
        Serial.println(F("Acerque una tarjeta..."));
    } else {
        Serial.println(F("ERROR: no hay comunicacion con el lector RC522"));
        Serial.println(F("Revise cableado (MISO, MOSI, SCK, SS, RST) y alimentacion 3.3 V"));
    }
}

void loop() {
    unsigned long ahora = millis();

    // --- Apagado automático de los LEDs (sin bloquear) ---
    if (verdeEncendido && (ahora - inicioVerde >= TIEMPO_LED)) {
        digitalWrite(LED_VERDE, LOW);
        verdeEncendido = false;
    }
    if (rojoEncendido && (ahora - inicioRojo >= TIEMPO_LED)) {
        digitalWrite(LED_ROJO, LOW);
        rojoEncendido = false;
    }

    // --- Lectura de tarjetas (siempre activa) ---
    if (!rfid.PICC_IsNewCardPresent()) return;

```

```

if (!rfid.PICC_ReadCardSerial()) return;

imprimirUID(rfid.uid);

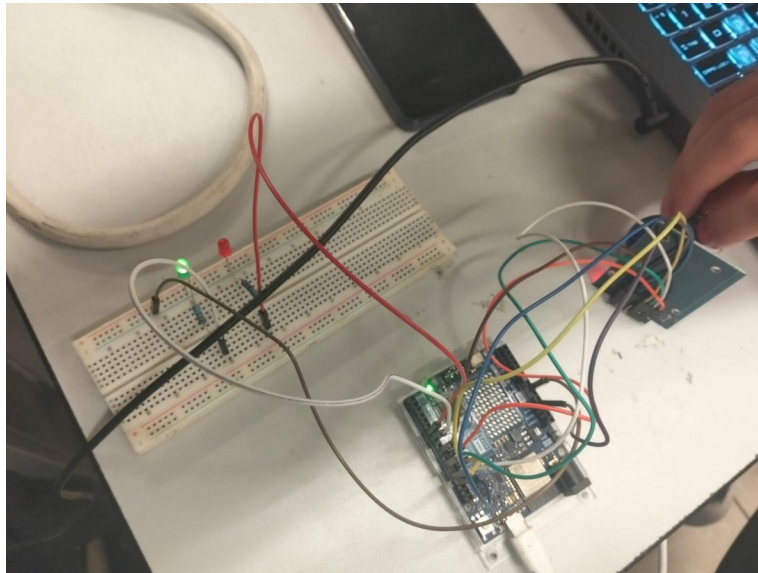
if (uidAutorizado(rfid.uid)) {
  Serial.println(F("ACCESO PERMITIDO"));
  digitalWrite(LED_VERDE, HIGH);
  verdeEncendido = true;
  inicioVerde = millis();          // reinicia el conteo de 2 s
  digitalWrite(LED_ROJO, LOW);    // apaga el rojo si estaba encendido
  rojoEncendido = false;
} else {
  Serial.println(F("ACCESO DENEGADO"));
  digitalWrite(LED_ROJO, HIGH);
  rojoEncendido = true;
  inicioRojo = millis();
  // El LED verde NO se toca: si estaba encendido, sigue su propio conteo
}

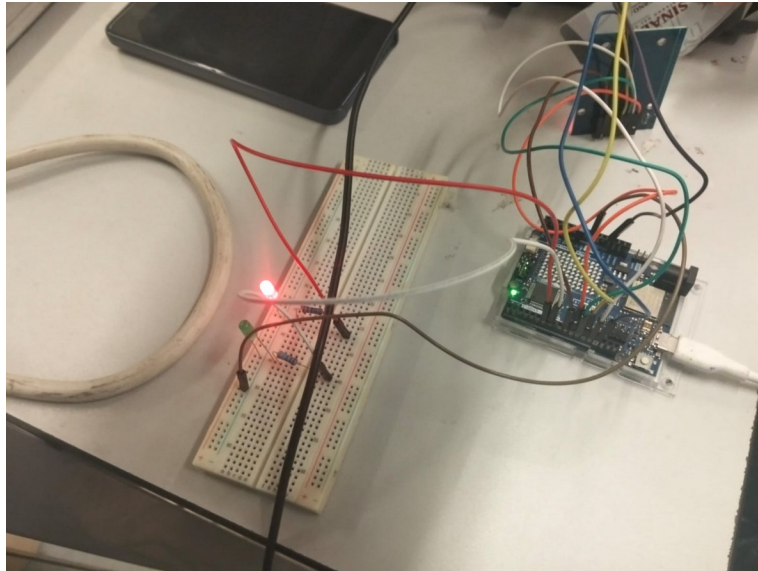
rfid.PICC_HaltA();                // detiene la tarjeta actual
rfid.PCD_StopCrypto1();           // termina la comunicación segura
}

```

6. Montaje físico

Como parte de la práctica se realizó el armado físico del circuito en protoboard, replicando las conexiones del diagrama y verificando de forma presencial el comportamiento del lector RC522.





Montaje físico del circuito.

7. Metodología

Primero se conectó el módulo RC522 al Arduino UNO R4 WiFi siguiendo el esquema del bus SPI (SCK, MOSI, MISO, SS y RST), alimentándolo con 3.3V. Posteriormente se conectaron los dos LEDs indicadores (verde y rojo), cada uno con su resistencia de 220Ω hacia GND.

En el código, se implementó una función que verifica la comunicación con el módulo leyendo su registro de versión (VersionReg); si el valor devuelto es 0x00 o 0xFF, se interpreta como una falla de comunicación SPI.

Se acercó una tarjeta o llavero RFID al lector, obteniendo su UID a través de la librería MFRC522, y se comparó contra un UID autorizado guardado previamente en el código. Según el resultado, se encendió el LED verde (acceso permitido) o el LED rojo (acceso denegado). El apagado de ambos LEDs, dos segundos después de encenderse, se controló con millis(), evitando detener la lectura del lector en ningún momento.

Como parte de las pruebas, se desconectó intencionalmente el cable MISO para observar el comportamiento del sistema ante una falla de comunicación, y posteriormente se volvió a conectar para confirmar que el sistema se recuperaba con normalidad.

8. Explicación del funcionamiento

El módulo RC522 se comunica con el Arduino mediante cuatro líneas del bus SPI, más una línea de reset y una de selección de esclavo. SCK (Serial Clock) es la línea de reloj que sincroniza el envío de cada bit entre el maestro y el esclavo. MOSI (Master Out, Slave In) es la línea por la que el Arduino, como maestro, envía datos hacia el RC522. MISO (Master In, Slave Out) es la línea por la que el RC522 responde enviando datos de regreso al Arduino. CS o SS (Chip/Slave Select) es la línea que el maestro pone en LOW para indicar con cuál esclavo específico se está comunicando, ya que varios dispositivos SPI pueden compartir las mismas líneas SCK, MOSI y MISO.

En la serigrafía del módulo, la línea de selección de esclavo aparece etiquetada como "SDA", lo cual puede generar confusión pensando que se trata de una comunicación I2C. En realidad, el chip MFRC522 puede operar en distintos modos de comunicación (SPI, I2C o UART) según cómo se configuren ciertos pines internos de fábrica, y el fabricante reutilizó esa etiqueta en la placa aunque en este módulo se está usando en modo SPI; el pin "SDA" no transmite datos I2C, cumple la función de SS/CS conectada al pin D10 del Arduino.

El UID de la tarjeta viaja del lector hacia el Arduino precisamente por la línea MISO, ya que es el dato que el RC522, como esclavo, envía de regreso hacia el Arduino en respuesta a la solicitud de lectura. El apagado de los LEDs se controla con millis() en lugar de delay(), porque delay() bloquearía por completo la ejecución del programa.

durante el tiempo indicado, dejando al Arduino sin capacidad de atender el bus SPI ni detectar nuevas tarjetas mientras el LED permanece encendido; si se hubiera usado `delay(2000)` tras cada lectura, el lector quedaría inactivo esos dos segundos y cualquier tarjeta acercada en ese lapso no sería detectada.

Un comportamiento identificado durante la práctica fue al desconectar el cable MISO: el Arduino siguió enviando comandos por MOSI, pero dejó de recibir cualquier respuesta del RC522. La función `lectorResponde()` leyó el registro de versión del chip y obtuvo `0x00` o `0xFF` —valores típicos de una línea flotante o sin comunicación—, por lo que el programa mostró el mensaje de error en el Monitor serie en lugar de fallar silenciosamente, confirmando que las cuatro líneas del bus SPI son igual de necesarias para una comunicación bidireccional.

9. Monitor Serie

Con el lector ya funcionando, el Monitor serie del Arduino muestra en tiempo real el UID leído en cada intento y el resultado de la validación de acceso (permitido o denegado):

```
UID: 01 35 D5 1C
ACCESO DENEGADO
UID: 01 35 D5 1C
ACCESO DENEGADO
UID: 01 35 D5 1C
ACCESO DENEGADO
UID: 8B 73 08 89
ACCESO DENEGADO
UID: 01 35 D5 1C
ACCESO PERMITIDO
```

Monitor Serie mostrando el UID leído y el resultado de acceso.

10. Preguntas de análisis

¿Qué significan las siglas SCK, MOSI, MISO y CS?

SCK (Serial Clock) es la línea de reloj que sincroniza el envío de cada bit entre el maestro y el esclavo. MOSI (Master Out, Slave In) es la línea por la que el maestro envía datos hacia el esclavo. MISO (Master In, Slave Out) es la línea por la que el esclavo envía datos de regreso al maestro. CS o SS (Chip/Slave Select) es la línea que el maestro pone en LOW para indicar con cuál esclavo específico se está comunicando, ya que varios dispositivos SPI pueden compartir las mismas líneas SCK, MOSI y MISO.

¿Por qué el pin del módulo dice "SDA" si no es I2C? ¿Qué función cumple en realidad?

Es una etiqueta heredada del diseño del módulo: el chip MFRC522 puede operar en distintos modos de comunicación (SPI, I2C o UART) según su configuración de fábrica, y el fabricante reutilizó el nombre "SDA" en la serigrafía aunque en este módulo se use en modo SPI. En la práctica, ese pin funciona como la línea de selección de esclavo (SS/CS) del bus SPI, conectada al pin D10 del Arduino.

¿Por qué línea viaja el UID del lector al Arduino?

El UID viaja por la línea MISO (Master In, Slave Out), ya que es el dato que el RC522 (esclavo) envía de regreso hacia el Arduino (maestro) como respuesta a la solicitud de lectura de la tarjeta.

¿Qué pasó al desconectar MISO y por qué?

Al desconectar MISO, el Arduino siguió enviando comandos por MOSI, pero ya no pudo recibir ninguna respuesta del RC522. La función `lectorResponde()` obtuvo `0x00` o `0xFF` al leer el registro de versión del chip (valores típicos de una línea flotante o sin comunicación), por lo que el programa reportó el error "no hay comunicación con el lector RC522". Ninguna tarjeta pudo leerse en ese estado, aunque el resto del cableado estuviera bien conectado.

¿Por qué no se permitió usar `delay()` en esta práctica? ¿Qué problema habría causado?

Se evitó `delay()` porque bloquea por completo la ejecución del programa durante el tiempo indicado, dejando al Arduino sin capacidad de atender el bus SPI ni detectar nuevas tarjetas. Si se hubiera usado `delay(2000)` para mantener encendido el LED, el lector quedaría inactivo durante esos 2 segundos, y cualquier tarjeta acercada en ese lapso no sería detectada, provocando que el sistema pierda eventos reales.

Si quisieran conectar un segundo lector RC522 al mismo Arduino, ¿qué cables compartirían y cuál tendría que ser diferente?

Compartirían las líneas del bus SPI comunes a todos los esclavos: SCK, MOSI, MISO, además de la alimentación (3.3V) y GND. La única línea que tendría que ser distinta para cada lector es la de selección de esclavo, SDA/SS (y opcionalmente RST), conectada a un pin digital distinto por cada RC522, de manera que el Arduino pueda elegir con cuál módulo comunicarse poniendo en LOW únicamente el SS correspondiente.

11. Análisis de resultados

Durante las pruebas se observó que el lector RC522 identificó correctamente el UID de cada tarjeta acercada, mostrándolo en el Monitor serie junto con el resultado de la validación. El sistema respondió de forma inmediata al acercar una tarjeta distinta a la autorizada, encendiendo el LED rojo, y al acercar la tarjeta correcta, encendiendo el LED verde.

El uso de `millis()` para controlar el apagado de los LEDs permitió que el lector continuara escuchando nuevas tarjetas en todo momento, sin los periodos "ciegos" que habría generado el uso de `delay()`. La prueba de desconexión de MISO confirmó la importancia de cada línea del bus SPI: sin esa línea, el sistema detecta y reporta la falla de comunicación en lugar de fallar silenciosamente.

12. Conclusiones

Gabriel

Esta práctica reforzó mi comprensión de cómo múltiples dispositivos pueden compartir un mismo bus de comunicación (SPI) y diferenciarse únicamente por su línea de selección de esclavo. Ver en la terminal cómo un UID no autorizado generaba "ACCESO DENEGADO" y el UID correcto generaba "ACCESO PERMITIDO" ayudó a conectar la teoría de comparación de datos con un resultado físico inmediato en los LEDs.

Javier

Trabajar con el lector RC522 me permitió aplicar el uso de `millis()` en un contexto distinto al de simples parpadeos de LED: aquí quedó claro por qué usar `delay()` habría sido un error, ya que el sistema necesita estar escuchando al lector en todo momento para no perder ninguna tarjeta. También aprendí a diagnosticar fallas de comunicación SPI leyendo el registro de versión del chip.

Rosa

Esta práctica me permitió comprender cómo funciona el protocolo SPI y la diferencia entre sus líneas de comunicación. Entender que el pin "SDA" del módulo en realidad es la selección de esclavo (SS) me ayudó a no confundir este protocolo con I2C, y comprobar qué pasa cuando una de las líneas (MISO) se desconecta reforzó la importancia de cada conexión dentro del bus.